

BE5210 Final Project Report

Team Three Braincells: Binoy Patel, Sohan Lele, Eric Ouyang

April 25, 2026

Summary

The final algorithm pipeline predicts individual finger flexion from ECoG recordings using an ensemble of two deep learning architectures implemented in TensorFlow/Keras. Each model was trained separately for each of the three subjects. We first preprocess the raw ECoG signals by applying common average referencing, clipping outliers using the median absolute deviation, bandpass filtering from 0.5 to 200Hz, and notch filtering at 60 and 120 Hz. The filtered signal is then segmented into non-overlapping 50 ms frames. From each frame, we extract 12 features per channel, including the mean time-domain voltage and log band power across 11 frequency bands spanning 0.5 to 170 Hz. The high-gamma range (70 to 170 Hz) is divided into five sub-bands for finer spectral resolution. The features are z-score normalized and arranged into sliding windows of 30 frames. This gives the model about 1.5 seconds of temporal context at each prediction step. For each subject, we train two models: (1) a ResNet-style CNN with residual blocks and global average pooling, and (2) a CNN paired with Bidirectional GRU to capture both local spatial patterns and longer-range temporal dependencies. Both models use Huber loss and AdamW optimization with gradient clipping. During prediction, we average the predictions of three models to produce the final output. Training labels are standardized to zero mean and unit variance, so the model treats all fingers equally. Finally, we apply per-finger Gaussian smoothing and interpolate the predictions back to 1000 Hz using cubic splines.

Step-by-Step Through the Algorithm

Step 1: Preprocessing

- Each subject is processed independently. We subtract the mean across channels to remove shared noise, clip extreme values beyond 6 scaled MADs from the median, apply a 4th-order Butterworth bandpass filter from 0.5 to 200 Hz, and notch filter at 60 and 120 Hz to remove powerline interference.

Step 2: Feature Extraction

- After filtering, the signal is broken into consecutive 50 ms windows with no overlap. We chose 50 ms rather than the 100 ms suggested in the assignment to double our temporal resolution. For each frame, we compute the mean voltage and log band power across 11 frequency bands from delta through high-gamma. The 70 to 170 Hz range is split into

five 20 Hz sub-bands. This is because high-gamma activity is the most informative frequency band for decoding motor commands from ECoG.

Step 3: Normalization and Sequence Construction

- Features are z-score normalized per subject. We then construct input sequences using a sliding window of 30 frames, padded at the start by repeating the first frame. This gives the model roughly 1.5 seconds of temporal history per prediction. We tried shorter windows but 30 consistently gave better correlation.

Step 4: Label Preparation

- DataGlove traces are downsampled to 50 ms by averaging within each frame and standardized to zero mean and unit variance. Without this, the model would over-optimize for high-amplitude fingers and underfit for the rest.

Step 5: Model Training

- For modeling, we used two architectures. The ResNet CNN uses three residual blocks (128, 192, 256 filters) with skip connections that help gradient flow during training, followed by global average pooling to compress the temporal dimension into a fixed-length vector. The CNN+BiGRU shares the same convolutional backbone but replaces pooling with a Bidirectional GRU (96 units per direction), which processes the sequence in both forward and backward directions to capture dependencies that the CNN alone might miss. Both use Huber loss, AdamW with weight decay and gradient clipping, early stopping, and learning rate reduction on plateau. Each is trained with 5 random seeds for 10 total candidate models per subject.

Step 6: Ensemble Selection

- A 95/5 train/validation split was used to maximize training data while retaining enough samples to reliably rank models. We keep the top 3 models per subject based on validation correlation across fingers 1, 2, 3, and 5. Then we average their predictions at inference. Ensembling consistently outperformed any single model by about 0.02 to 0.05 in correlation on our validation set.

Step 7: Post-Processing and Upsampling

- The ensemble output is first de-standardized using the saved training label statistics, then smoothed using per-finger Gaussian filters. Higher smoothing for slower fingers (thumb or little finger) and less for faster fingers (middle finger). Predictions are then interpolated from 20 Hz back to 1000 Hz using cubic spline interpolation to avoid edge artifacts.

Flow Chart

Our final pipeline can be summarized as:

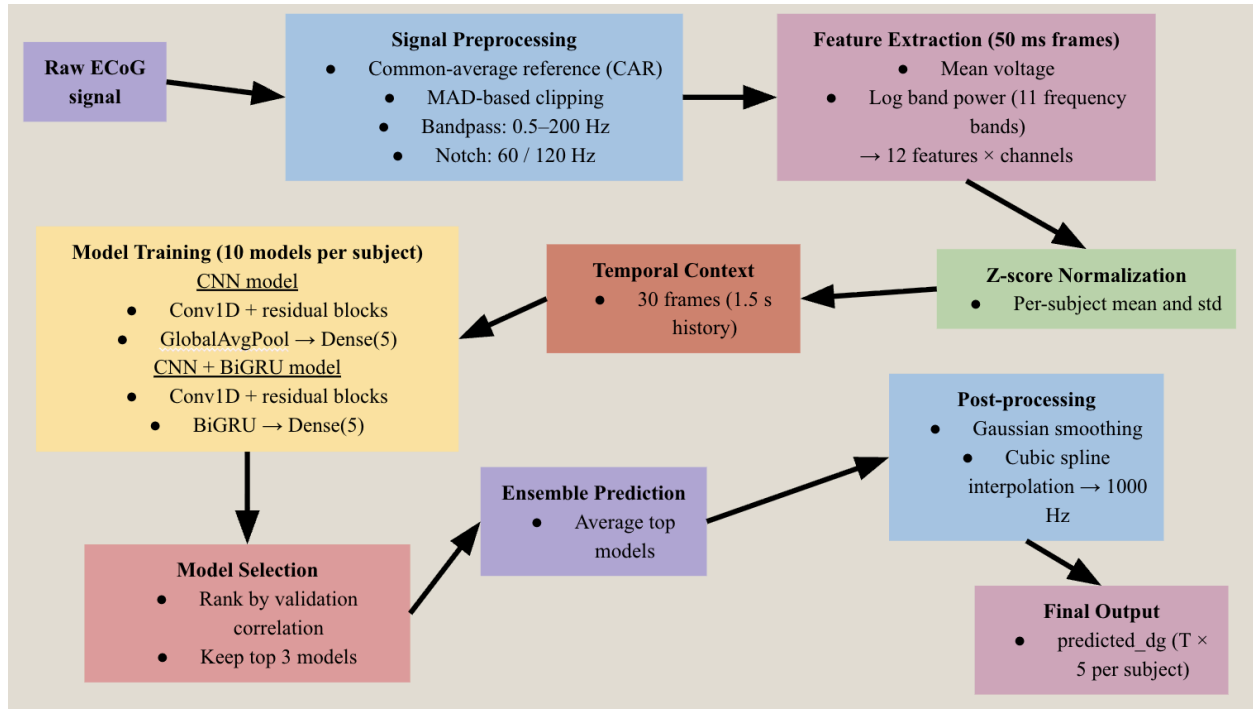


Figure 1: Overview of the final algorithm pipeline

Figures

Figure 2 illustrates the performance of our final model on Subject 1. The figure shows the mean correlation achieved by several models, such as two CNN variants, a CNN-GRU model, and a final ensemble based on results while training. While individual models achieved correlations in the range of around 0.464–0.530, the ensemble improved to 0.568, much higher than the models on their own. This demonstrates that combining multiple strong models led to a consistent performance boost over any single model.

The lower figure, Figure 3, shows predicted versus actual finger flexion traces for all five fingers on a validation segment. The model captures the overall timing of the movements well, especially for the thumb and index fingers. This is where the predicted peaks line up with the ground truth even though the predicted amplitudes are attenuated. For the middle, ring, and little fingers, the model has more difficulty and produces small extra peaks where the actual signal is mostly flat. This reflects our algorithm’s tendency to underpredict large flexions while still being noisy on the weaker, more correlated fingers.

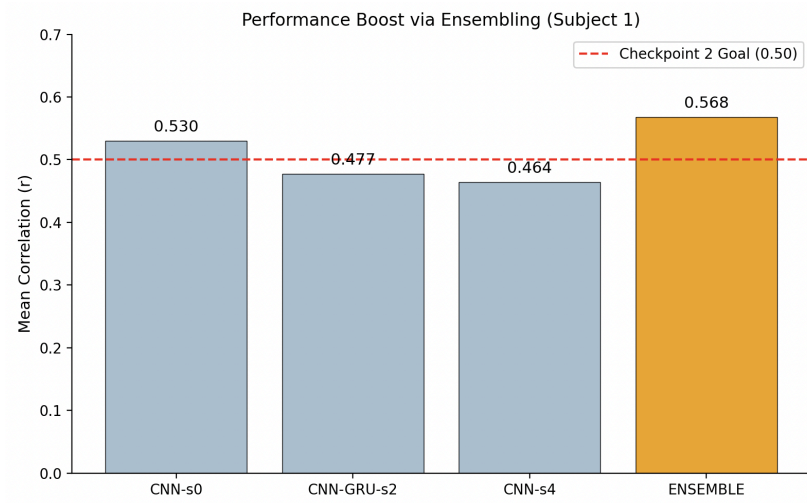


Figure 2: Different Model Performances vs Ensemble

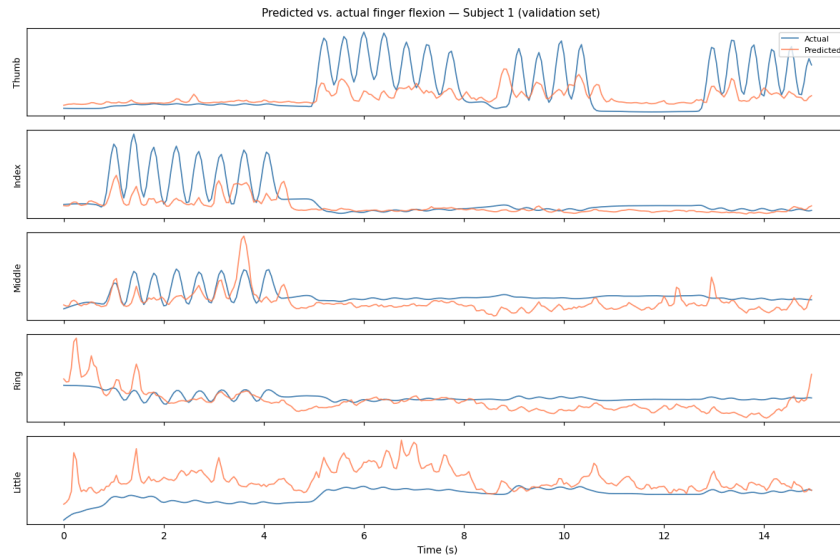


Figure 3: Predicted vs Actual on Validation Set

Methods that did not Work

Our first approach was the simple method from the project instructions (Kubanek et al., 2009). We extracted 6 features per ECoG channel from 100ms sliding windows with 50ms overlap. These included the average time-domain voltage and mean spectral magnitude in five frequency bands. We then built a lagged response matrix with $N_{\text{wind}} = 3$ to capture 150ms of temporal history. Predictions were interpolated back to the original 1000 Hz sampling rate using cubic spline interpolation. This achieved around $r = 0.24$ on our 80/20 split, well below the $r > 0.35$

checkpoint. The middle and little fingers had some correlations near zero or negative and were especially weak.

We also tried to implement Ridge regression as a regularized alternative, but got identical results. The problem was that the default $\alpha = 1.0$ was negligibly small relative to the scale of RTR. Hence, regularization had no meaningful effect on the solution. We tried several improvements to boost performance, including adding variance as a 7th feature, applying a log transform to the frequency band power features, z-score standardizing the feature matrix, increasing N_{wind} from 3 to 5, and using RidgeCV with cross-validated alpha selection. These improvements resulted in better performance, and Ridge finally outperformed OLS. However, the linear model still couldn't consistently get above $r > 0.35$. Ultimately, a linear model wasn't powerful enough to capture the complex relationship between ECoG signals and individual finger movements. This is why we decided to switch to a CNN-based approach.

Ring Finger

The ring finger is generally highly correlated with the middle and little fingers because it is mechanically and neurally tied with other fingers. From a biomechanical perspective, the ring finger shares muscles and tendon connections with adjacent fingers, making it harder to move in complete isolation. Because of this coupling, ring finger flexion often occurs with movement in the middle or little finger, even when the intended action is not perfectly identical.

From a neural perspective, the cortical representations for neighboring fingers in motor areas are not completely separate, so ECoG activity associated with one finger can overlap with activity related to nearby fingers. That means the neural signals used to decode the ring finger may already contain information about the middle and little fingers, and vice versa. This is likely why the fourth finger is excluded from the evaluation metric.

Conclusion

In conclusion, this final project gave us a much deeper understanding and appreciation for how complex the relationship between brain signals and movement actually is. What surprised us most was how much every stage of the pipeline mattered. A poor processing step or even the wrong smoothing parameter was enough to significantly hurt performance, regardless of how good the model was. Getting all the components to work together correctly was just as important as choosing the right model.

We moved well past the linear regression baseline by incorporating richer frequency features, a longer temporal context, and an ensemble of deep learning models tailored to each subject individually. In terms of performance, our final model achieved a leaderboard score of $r=0.5437$, comfortably exceeding the checkpoint 2 threshold. In the end, this project was a good example of

how performance came from iterative refinement rather than from one single trick, and it gave us a much better understanding of how neural signals can be translated into motor behavior. If we had more time, we would have explored transformer-based architectures and more extensive hyperparameter search. We believe this would've likely pushed our correlation scores even higher.

References

Kubaneck, J., Miller, K. J., Ojemann, J. G., Wolpaw, J. R., and Schalk, G. “Decoding flexion of individual fingers using electrocorticographic signals in humans.” *Journal of Neural Engineering*, 2009.

Warland, D. K., Reinagel, P., and Meister, M. “Decoding visual information from a population of retinal ganglion cells.” *J. Neurophysiol.* 78: 2336–2350, 1997.

Freund, Y., and Schapire, R. E. “A decision-theoretic generalization of on-line learning and an application to boosting.” European Conference on Computational Learning Theory (EuroCOLT), 1995.

TensorFlow / Keras documentation for implementation details of convolutional and GRU-based neural networks.

SciPy documentation for signal filtering, Gaussian smoothing, and cubic-spline interpolation.

Appendix

Appendix A contains the final version of the code used for preprocessing, feature extraction, model training, ensemble selection, leaderboard prediction, smoothing, interpolation, and .mat file generation. Only the final code used in the submitted pipeline is included here, as required by the project instructions. We used AI to help debug errors in the code, explain things conceptually, and brainstorm new ideas.

Cell 1

```
#imports
import numpy as np
import scipy.io
from scipy import signal as sig
from scipy.interpolate import CubicSpline
from scipy.stats import pearsonr
from scipy.ndimage import gaussian_filter1d
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

Cell 2

```
# load training ECoG + data glove and leaderboard ECoG for all 3 subjects
train_data = scipy.io.loadmat('raw_training_data.mat')
lead_data = scipy.io.loadmat('leaderboard_data.mat')

train_ecog = [train_data['train_ecog'][i, 0].astype(np.float32) for i in range(3)]
train_dg = [train_data['train_dg'][i, 0].astype(np.float32) for i in range(3)]
lead_ecog = [lead_data['leaderboard_ecog'][i, 0].astype(np.float32) for i in range(3)]
```

```

for i in range(3):
    print(f'Subject {i+1}: ECoG {train_ecog[i].shape}, DG {train_dg[i].shape}')
print('Leaderboard:', [e.shape for e in lead_ecog])

```

Cell 3

```

# Preprocessing: bandpass 0.5-200 Hz, remove outliers with MAD, notch filter 60/120
Hz
#then extract log band power features in 50ms windows across 11 frequency bands
def _get_bandpass(f_low, f_high, sample_rate=1000, filt_order=4):
    return sig.butter(filt_order, [f_low, f_high], btype='bandpass', fs=sample_rate,
output='sos')

def filter_data(data, fs=1000):
    data = data.astype(np.float32)
    data = data - data.mean(axis=1, keepdims=True)
    channel_median = np.median(data, axis=0, keepdims=True)
    median_abs_dev = np.median(np.abs(data - channel_median), axis=0, keepdims=True)
    + 1e-6
    threshold = 6 * 1.4826 * median_abs_dev
    data = np.clip(data, channel_median - threshold, channel_median + threshold)
    cleaned = sig.sosfiltfilt(_get_bandpass(0.5, 200, sample_rate=fs, filt_order=4),
data, axis=0)

    for notch_freq in [60, 120]:
        b, a = sig.iirnotch(notch_freq, Q=30, fs=fs)
        cleaned = sig.filtfilt(b, a, cleaned, axis=0)
    return cleaned.astype(np.float32)

FREQ_BANDS = [
    (0.5, 4), (4, 8), (8, 13), (13, 30), (30, 50),
    (50, 70), (70, 90), (90, 110), (110, 130),
    (130, 150), (150, 170)
]

def compute_band_features(filtered, fs=1000, frame_ms=50):
    win_len = int(fs * frame_ms / 1000)
    n_samples, num_channels = filtered.shape
    num_frames = n_samples // win_len
    windowed = filtered[:num_frames * win_len].reshape(num_frames, win_len,
num_channels)
    feature_list = [windowed.mean(axis=1)]

    for f_low, f_high in FREQ_BANDS:
        bp_sos = _get_bandpass(f_low, f_high, sample_rate=fs, filt_order=4)
        band_signal = sig.sosfiltfilt(bp_sos, filtered, axis=0)
        band_signal = band_signal[:num_frames * win_len].reshape(num_frames,
win_len, num_channels)
        feature_list.append(np.log(np.mean(band_signal ** 2, axis=1) + 1e-4))
    all_features = np.concatenate(feature_list, axis=1).astype(np.float32)

    return np.nan_to_num(all_features, nan=0.0, posinf=0.0, neginf=0.0), win_len

```


Cell 4

```

# build sliding window sequences (30 frames of context) for the model input
# also downsamples the dg to match the feature frame rate during training
CONTEXT_FRAMES = 30
FRAME_SAMPLES = 50

def build_sequences(features, dg=None, context=CONTEXT_FRAMES):
    n_frames, F = features.shape
    pad = np.repeat(features[:,1], context - 1, axis=0)
    padded = np.concatenate([pad, features], axis=0)
    X = np.lib.stride_tricks.sliding_window_view(
        padded, window_shape=context, axis=0
    ).transpose(0, 2, 1)[:n_frames]
    if dg is not None:
        Y = np.zeros((n_frames, 5), dtype=np.float32)
        for i in range(n_frames):
            s = i * FRAME_SAMPLES
            Y[i] = dg[s:s+FRAME_SAMPLES].mean(axis=0)
        return X.astype(np.float32), Y
    return X.astype(np.float32)

```

Cell 5

```

# two model architectures: ResNet-style CNN and CNN+BiGRU hybrid
# both output 5 values, one per finger, and use Huber loss
def residual_block(inp, n_filters, kernel=3, drop_rate=0.0):
    r = layers.Conv1D(n_filters, kernel, padding='same', use_bias=False)(inp)
    r = layers.BatchNormalization()(r)
    r = layers.ReLU()(r)

    if drop_rate > 0:
        r = layers.SpatialDropout1D(drop_rate)(r)
    r = layers.Conv1D(n_filters, kernel, padding='same', use_bias=False)(r)
    r = layers.BatchNormalization()(r)

    if inp.shape[-1] != n_filters:
        inp = layers.Conv1D(n_filters, 1, padding='same', use_bias=False)(inp)
    return layers.ReLU()(layers.Add()([inp, r]))

def build_cnn(n_features):
    input_layer = keras.Input(shape=(CONTEXT_FRAMES, n_features))
    x = layers.Conv1D(128, 3, padding='same', activation='relu')(input_layer)
    x = layers.BatchNormalization()(x)
    x = residual_block(x, 128)
    x = residual_block(x, 192)
    x = layers.Dropout(0.2)(x)
    x = residual_block(x, 256)
    x = layers.GlobalAveragePooling1D()(x)
    x = layers.Dense(128, activation='relu')(x)
    x = layers.Dropout(0.3)(x)
    output = layers.Dense(5)(x)

```

```

    model = keras.Model(input_layer, output)
    model.compile(
        optimizer=keras.optimizers.AdamW(learning_rate=8e-4,
weight_decay=1e-4, clipnorm=1.0),
        loss='huber'
    )
    return model

def build_cnn_gru(n_features):
    input_layer = keras.Input(shape=(CONTEXT_FRAMES, n_features))
    x = layers.Conv1D(128, 3, padding='same', activation='relu')(input_layer)
    x = layers.BatchNormalization()(x)
    x = residual_block(x, 128, drop_rate=0.15)
    x = residual_block(x, 192, drop_rate=0.15)
    x = layers.Dropout(0.35)(x)
    x = residual_block(x, 256, drop_rate=0.15)
    x = layers.Bidirectional(layers.GRU(96, return_sequences=False,
dropout=0.2))(x)
    x = layers.Dense(128, activation='relu')(x)
    x = layers.Dropout(0.45)(x)
    output = layers.Dense(5)(x)
    model = keras.Model(input_layer, output)
    model.compile(
        optimizer=keras.optimizers.AdamW(learning_rate=8e-4,
weight_decay=1e-4, clipnorm=1.0),
        loss='huber'
    )
    return model

```

Cell 6

```

# train 10 models per subject (5 seeds x 2 architectures), keep the best 3
# evaluate using pearson correlation on fingers 1,2,3,5 skip 4
SEEDS_PER_ARCH = 5
KEEP_TOP = 3
EPOCHS = 70
BATCH = 256
VAL_FRAC = 0.05

ensembles = []

for subj in range(3):
    print(f'\n--- Subject {subj+1} ---')
    filtered = filter_data(train_ecog[subj])
    F, _ = compute_band_features(filtered)
    print(f' Features shape: {F.shape}')

    T_frames = F.shape[0]
    split = int((1 - VAL_FRAC) * T_frames)
    f_mean = F.mean(0, keepdims=True)
    f_std = F.std(0, keepdims=True) + 1e-6
    F_norm = (F - f_mean) / f_std

```

```

X, Y = build_sequences(F_norm, train_dg[subj])
X_tr, Y_tr = X[:split], Y[:split]
X_va, Y_va = X[split:], Y[split:]

y_mean = Y_tr.mean(0, keepdims=True)
y_std = Y_tr.std(0, keepdims=True) + 1e-6
Y_tr_n = (Y_tr - y_mean) / y_std
Y_va_n = (Y_va - y_mean) / y_std

all_models, all_scores, all_names = [], [], []

for arch_name, builder in [('cnn', build_cnn), ('cnn_gru',
build_cnn_gru)]:
    for seed in range(SEEDS_PER_ARCH):
        tag = f'{arch_name}-s{seed}'
        print(f' -- training {tag}')
        keras.utils.set_random_seed(seed + (0 if arch_name == 'cnn' else
100))
        m = builder(F.shape[1])
        cbs = [
            keras.callbacks.EarlyStopping(patience=12,
restore_best_weights=True, monitor='val_loss'),
            keras.callbacks.ReduceLROnPlateau(factor=0.5,
patience=6, min_lr=1e-5),
        ]
        m.fit(X_tr, Y_tr_n, validation_data=(X_va, Y_va_n),
            epochs=EPOCHS, batch_size=BATCH, callbacks=cbs, verbose=0)
        p = m.predict(X_va, verbose=0) * y_std + y_mean
        corrs = [pearsonr(Y_va[:, f], p[:, f])[0] for f in range(5)]
        score = np.mean([corrs[0], corrs[1], corrs[2], corrs[4]])
        print(f' {tag} corrs={corrs} avg={score:.3f}')
        all_models.append(m); all_scores.append(score);
all_names.append(tag)

order = np.argsort(all_scores)[::-1][:KEEP_TOP]
kept_models = [all_models[i] for i in order]
kept_names = [all_names[i] for i in order]
print(f'\n Kept top-{KEEP_TOP}: {kept_names}')

preds = np.mean([m.predict(X_va, verbose=0) for m in kept_models], axis=0)
* y_std + y_mean
corrs = [pearsonr(Y_va[:, f], preds[:, f])[0] for f in range(5)]
score = np.mean([corrs[0], corrs[1], corrs[2], corrs[4]])
print(f' Ensemble corrs: {corrs} avg={score:.3f}')

ensembles.append({'models': kept_models, 'f_mean': f_mean, 'f_std': f_std,
'y_mean': y_mean, 'y_std': y_std})

```

Cell 7

```
# save trained models and normalization params
```

```

import os, zipfile, pickle

os.makedirs('saved_models', exist_ok=True)
save_data = []

for subj, ens in enumerate(ensembles):
    subj_dir = f'saved_models/subject{subj+1}'
    os.makedirs(subj_dir, exist_ok=True)
    model_paths = []
    for mi, m in enumerate(ens['models']):
        path = f'{subj_dir}/model_{mi}.keras'
        m.save(path)
        model_paths.append(path)
        print(f'  Saved subject {subj+1} model {mi}: {path}')
    save_data.append({
        'model_paths': model_paths,
        'f_mean': ens['f_mean'], 'f_std': ens['f_std'],
        'y_mean': ens['y_mean'], 'y_std': ens['y_std'],
    })

with open('ensemble_params.pkl', 'wb') as f:
    pickle.dump(save_data, f)
print('Saved ensemble_params.pkl')
with zipfile.ZipFile('submission_models.zip', 'w', zipfile.ZIP_DEFLATED) as zf:
    for subj in range(3):
        for mi in range(len(ensembles[subj]['models'])):
            path = f'saved_models/subject{subj+1}/model_{mi}.keras'
            zf.write(path)
print('Created submission_models.zip')

```

Cell 8

```

# download locally
from google.colab import files
files.download('submission_models.zip')
files.download('ensemble_params.pkl')

```

Cell 9

```

#run ensemble on leaderboard data, smooth predictions, and interpolate back to
1000 Hz using cubic splines
SMOOTH_SIGMAS = [4.0, 3.0, 2.5, 3.0, 4.0]
predicted_dg = []

for subj in range(3):
    ens = ensembles[subj]
    filt = filter_data(lead_ecog[subj])
    feat, frame_len = compute_band_features(filt)

```

```

    feat_norm = np.nan_to_num((feat - ens['f_mean']) / ens['f_std'], nan=0.0,
                               posinf=0.0, neginf=0.0)
    X = build_sequences(feat_norm)

    raw_preds = np.mean([m.predict(X, batch_size=512, verbose=0) for m in
                          ens['models']], axis=0)
    raw_preds = raw_preds * ens['y_std'] + ens['y_mean']
    raw_preds = np.nan_to_num(raw_preds, nan=0.0, posinf=0.0, neginf=0.0)

    # smooth each finger's prediction
    smoothed = np.stack([gaussian_filter1d(raw_preds[:, f],
                                           sigma=SMOOTH_SIGMAS[f]) for f in range(5)], axis=1)

    n_lead_samples = filt.shape[0]
    frame_centers = np.arange(smoothed.shape[0]) * frame_len + frame_len / 2
    t_out = np.arange(n_lead_samples)
    interp_dg = np.zeros((n_lead_samples, 5), dtype=np.float32)

    for i in range(5):
        t_nodes = np.concatenate([[0], frame_centers, [n_lead_samples - 1]])
        y_nodes = np.concatenate([[smoothed[0, i]], smoothed[:, i],
                                   [smoothed[-1, i]]])
        cs = CubicSpline(np.nan_to_num(t_nodes), np.nan_to_num(y_nodes),
                         bc_type='not-a-knot')
        interp_dg[:, i] = cs(t_out).astype(np.float32)

    predicted_dg.append(interp_dg)
    print(f'Subject {subj+1}: {interp_dg.shape}')

```

Cell 10

```

# save predictions as a 3x1 cell array .mat file for leaderboard submission
out = np.empty((3, 1), dtype=object)
for i in range(3):
    out[i, 0] = predicted_dg[i]

scipy.io.savemat('predicted_dg.mat', {'predicted_dg': out})
print('Saved predicted_dg.mat')

```